

Final Report: Wall Construction Simulation Project

Collaborateurs : JULIANS Jean-yann , LEBAMA Jacques, Wahaga Clément

TABLE OF CONTENTS

Table of Contents Page

1. Implementation Description

1.1. Explanation of Class Roles

1.2. Synchronization Mechanisms

1.3. Design Choices

2. Real-Time Execution Screenshot

2.1. Wall Construction Steps

2.2. Random Events

3. Performance Analysis and Optimizations

3.1. Challenges in Managing Concurrency

3.2. Optimization Solutions

4. Discussion of Random Events

4.1. Impact of Random Events on Construction

Implementation Description

The Wall Construction Simulation Project was designed to emulate a collaborative wall-building process with multiple workers. This implementation uses multithreading in Java to simulate real-world concurrency and synchronization challenges. Below is an in-depth description of the components and their functionality:

Wall Class

The Wall class serves as the core of the simulation, managing the construction progress and coordinating threads. It includes the following features:

- State Management:
 - Tracks the number of rows (rows), columns (cols), and total stones placed (stonesPlaced).
 - Maintains a currentRow variable to indicate the progress across rows.
 - Uses a constructionComplete flag to signal when the wall is fully built.
- Synchronization Mechanisms:
 - Utilizes synchronized methods and the wait() and notifyAll() functions to ensure proper thread coordination.
 - Threads are blocked using wait() when the wall is under construction and notified simultaneously with notifyAll() upon completion.
- Progress Tracking:
 - After each stone is placed, the class calculates and displays the wall's progress in terms of percentage.
 - Provides detailed output including row and column coordinates for each stone placed.

StoneLayer Class

The StoneLayer class represents the workers who place stones on the wall. Key features include:

- Task Execution:
 - Continuously calls the placeStone() method in the Wall class to add stones until the wall is complete.

- Includes error handling to ensure threads gracefully terminate when construction is completed.
- Synchronization Handling:
 - Leverages `Wall.waitForCompletion()` to synchronize with other threads, ensuring a coordinated stop when the wall is finished.

Synchronization Mechanisms

To prevent race conditions and ensure smooth operations, the program employs:

- `ReentrantLock`:
 - Used to lock critical sections, preventing multiple threads from accessing shared variables concurrently.
- `notifyAll()` and `wait()`:
 - Used to synchronize threads, ensuring all workers stop simultaneously when the wall construction is complete.
- Thread-safe Variables:
 - The `constructionComplete` flag and `stonesPlaced` variable are accessed using synchronized methods to maintain consistency.

Design Choices

- Lightweight Synchronization:
 - Only critical sections are synchronized to minimize performance overhead.
- Thread Communication:
 - Threads use the `wait()` and `notifyAll()` mechanism for efficient coordination.
- Realistic Simulation:
 - Delays (`Thread.sleep()`) are added to simulate realistic worker actions.

Real-Time Execution Screenshot

The program generates detailed outputs for every action, showing:

- Stone placement events with worker identification.
- Row and column coordinates of placed stones.
- Current wall progress in percentage.

```
Transporter-1 placed a stone at Row: 88, Col: 1. Total: 262
Wall progress: 87,33% (262/300 stones placed)
Transporter-0 placed a stone at Row: 88, Col: 2. Total: 263
Wall progress: 87,67% (263/300 stones placed)
Mason-0 placed a stone at Row: 88, Col: 3. Total: 264
Wall progress: 88,00% (264/300 stones placed)
Row 88 completed!
Mason-1 placed a stone at Row: 89, Col: 1. Total: 265
Wall progress: 88,33% (265/300 stones placed)
```

- Final message indicating completion: "Wall construction is complete!"

```
Mason-0 is stopping as the wall is completed.
Mason-0 is stopping as the wall is completed.
Mason-1 is stopping as the wall is completed.
Mason-1 is stopping as the wall is completed.
Mason-1 is stopping as the wall is completed.
Mason-1 is stopping as the wall is completed.
Mason-1 is stopping as the wall is completed.
Mason-1 is stopping as the wall is completed.
```

Performance Analysis and Optimizations

Concurrency Challenges

- Race Conditions:
 - Multiple threads accessing and modifying shared variables could lead to inconsistent states.
- Thread Coordination:
 - Ensuring all threads terminate simultaneously when the wall is complete required robust synchronization.

Optimizations

- Efficient Synchronization:
 - Reduced the scope of synchronized blocks to improve thread efficiency.
- Scalable Thread Management:
 - Added mechanisms to gracefully handle a large number of worker threads without excessive contention.
- Minimal Overhead:

- The combination of ReentrantLock and synchronized methods ensures high performance without sacrificing safety.

Discussion of Random Events

To simulate real-world construction processes, random delays and interruptions were introduced:

- Impact of Random Delays:
 - Simulated differences in worker efficiency, creating variability in the order and speed of stone placement.
- Resilience to Uncertainty:
 - Synchronization mechanisms ensured that despite delays, the overall construction process remained consistent and robust.
- Thread Interruption Handling:
 - Threads gracefully stopped upon wall completion, preventing deadlocks or unresponsive states.

Conclusion

The Wall Construction Simulation Project successfully demonstrates the use of multithreading, synchronization, and concurrency control in Java. With robust synchronization mechanisms in place, the simulation effectively handles multiple worker threads, ensuring coordinated progress and a smooth stop upon completion. The program provides a detailed real-time view of the construction process, making it an excellent demonstration of concurrency principles in action.